Technical Report: Agent Marketplace Backend

Module: COMP3011 Web Services and Web Data
Artefact: Technical report for the Agent Marketplace Backend repository
Public GitHub repository: https://github.com/FJPez/agent-marketplace
API documentation (PDF): https://github.com/FJPez/agent-marketplace/blob/main/docs/api-reference.pdf
Live deployment URL: https://api-production-b705.up.railway.app
Presentation Slides: https://docs.google.com/presentation/d/1O4cJ9IYXcOrrxucqrTvgFnUAQfciI4e7zgBCn5c5uDM/edit?usp=sharing

1. Introduction and Project Scope

This project implements a backend-only marketplace in which providers publish callable services and consumers discover, quote, invoke, and optionally pay for access to them. The system is aimed at both autonomous agents and human operators, so the design prioritises explicit HTTP contracts, machine-readable schemas, predictable error handling, and auditable persistence. Rather than acting as a simple directory, the platform combines registry, discovery, invocation, payment enforcement, moderation, and reporting responsibilities in one service.

The core operational lifecycle is `publish -> discover -> quote -> invoke -> pay -> execute -> record`. This lifecycle shaped the report and the implementation: a provider must be able to define a service safely, a consumer must be able to inspect that service before calling it, and the platform must only forward paid traffic after payment state is verified. The repository README and API reference are therefore central evidence in this report because they expose the current runtime behaviour rather than historical planning intent.

2. Requirements Compliance Snapshot

The coursework brief asks for a database-backed web API, at least four endpoints, appropriate JSON responses and status codes, documentation, version control evidence, and demonstrable execution. Table 1 maps those requirements to repository evidence.

Brief requirement	Repository evidence
Database-backed API	PostgreSQL is the persistence layer, modelled through SQLAlchemy async ORM with Alembic migrations under `alembic/versions/`.
Four or more HTTP endpoints	The current route surface contains 39 route handlers across the root entrypoint, auth, account, provider, discovery, quotes, invoke, finance, admin, and health.
JSON responses and industry status codes	FastAPI response models and route tests cover `200`, `201`, `204`, `401`, `403`, `404`, `409`, `413`, `422`, `429`, `500`, and `402 Payment Required`.
Clear API documentation	The repository contains both `docs/api-reference.md` and a committed PDF version.
Visible version control	The public Git history contains 242 commits, which provides clear evidence of iterative development.
Demonstrable execution	The project is locally runnable via `uv`, includes demo scripts, and contains a concrete Railway deployment runbook with health checks and smoke tests.

The brief also asks for CRUD. In this repository, CRUD behaviour is distributed across persisted resources rather than exposed as one textbook CRUD resource. For example, provider services support create, read, and update flows; accounts support read and update flows; API keys support create, read, and delete flows; and the database layer persists additional entities such as quotes, invocations, payment attempts, ledger entries, and payouts. I therefore present CRUD capability honestly as a repository-wide persistence pattern rather than overstating a single-resource CRUD surface.

3. Technology Stack and Rationale

Python 3.12 was selected because it provides a mature ecosystem for typed asynchronous web services and is already well supported by the module's recommended tooling. FastAPI was a strong fit because it combines automatic request parsing, OpenAPI-friendly route definitions, dependency injection, and explicit status-code handling with relatively little framework overhead [1]. This is important for an API-first backend where transport clarity matters as much as business logic.

Pydantic v2 and SQLAlchemy 2.x async were chosen to separate transport contracts from persistence models cleanly. Pydantic provides strict request and response schemas, while SQLAlchemy async gives control over transactions, relationships, and JSON-capable database fields without exposing ORM objects directly to clients [2][3]. PostgreSQL was the right database for this project because the marketplace combines transactional data, relational integrity, and JSON request or response payloads, all of which PostgreSQL handles well [5]. Alembic complements this by giving migration discipline and a traceable schema history [4].

The rest of the stack supports engineering quality rather than novelty for its own sake. `uv` simplifies environment reproducibility, pytest and httpx support layered testing, Ruff and `ty` enforce code quality, Redis-backed guardrails protect invoke and quote traffic, and x402 provides a machine-to-machine payment model that is particularly relevant for agent systems [6]. This was an important choice because autonomous agents cannot realistically rely on traditional consumer payment instruments such as credit cards during automated API invocation. Instead, they need a programmable payment flow that can be initiated, verified, and settled as part of the request lifecycle itself. No external dataset is used in this project because the system is a service marketplace rather than an analytics platform built around imported public data.

4. Architecture and Data Design

The codebase follows a modular-monolith structure. Route handlers remain thin and focus on HTTP concerns, service classes own orchestration and business rules, repositories encapsulate database access, and integrations isolate external protocols such as x402 and provider forwarding. This layered structure is appropriate for coursework because it demonstrates software engineering discipline without introducing the deployment and operational overhead of microservices.

The domain model is split into identity, service definition, contract tracking, commerce, and operations. Identity includes accounts, API keys, and wallet change logs. Service definition includes provider services, tags, endpoints, upstreams, and pricing. Contract tracking is handled through service revisions and change tokens. Commerce includes quotes, invocations, payment attempts, ledger entries, and payouts. Operations adds moderation actions and health checks. This model supports auditable state changes, hidden upstream routing, and clear separation between externally visible contracts and internal execution details.

A key design decision in this project was the mutability model for provider-owned services. While a service is still in draft, broad mutation is allowed because the contract has not yet been offered publicly: metadata, tags, endpoint definitions, schemas, pricing, upstream configuration, and timeout settings can all be refined as the provider authors the service. After publication, the model becomes more restrictive. Non-contract metadata such as descriptive text can still change, but contract-affecting fields such as request schema, response schema, pricing, access mode, timeout, and endpoint activation state should not change silently. Stable identity fields such as service slug, endpoint key, ownership, historical revisions, quotes, invocations,

and financial records are treated as effectively immutable because changing them would undermine traceability and trust.

Figure 1. High-level architecture

Two design choices are especially important. First, provider upstream configuration is stored privately and never leaked through public response models, which protects providers and keeps the marketplace in control of paid routing. Second, contract-affecting changes are tied to revisions and change tokens, so what a consumer pays for cannot silently change between quote creation and invoke time.

The revision contract exists to enforce that rule in a concrete way. When a published service changes in a way that affects what a consumer is buying, the system records a new revision snapshot and bumps a change token. Quotes and invoke requests can then be bound to a specific contract state rather than to a moving target. This is needed because the marketplace handles priced access: without revision-aware contract tracking, a consumer could request a quote for one schema or pricing model and then invoke against a different one after the provider had edited the service. The revision model therefore supports stale-quote rejection, reproducibility, and a stronger audit trail for both technical debugging and commercial correctness.

5. API Design, Security, and Runtime Behaviour

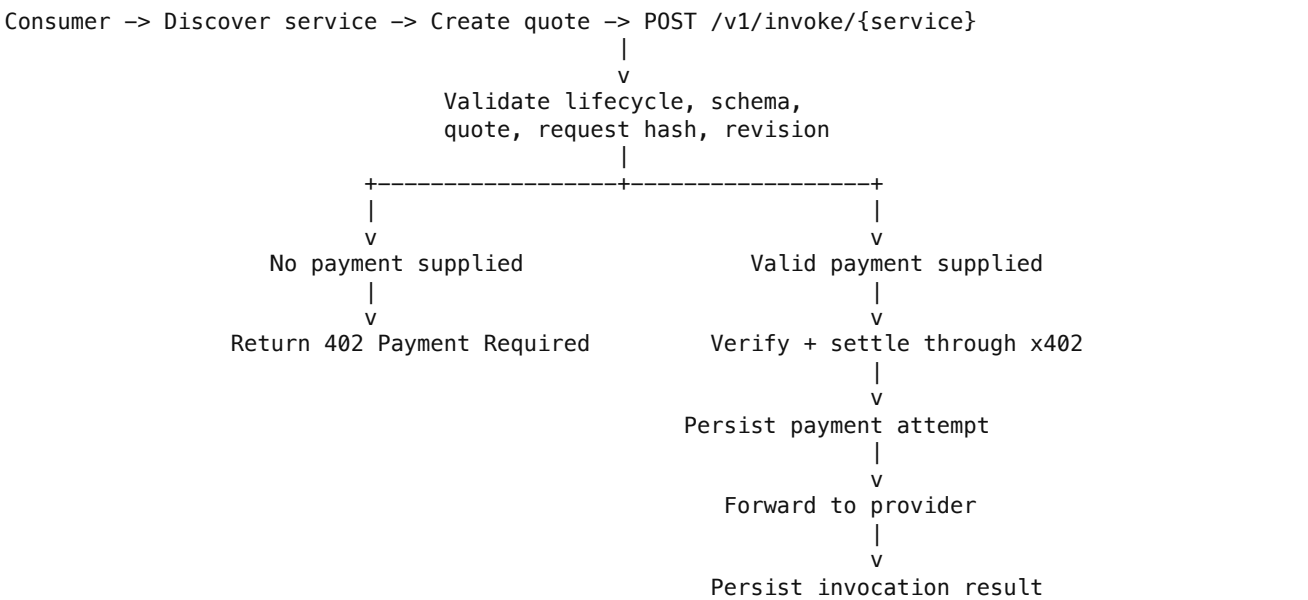
The route surface is deliberately broad because the platform covers the full marketplace lifecycle. Public routes support discovery, pricing, schema inspection, quote creation, and health checks. Authenticated routes support provider authoring, invocation, finance reporting, and moderation. This is more realistic than a minimal coursework API because it demonstrates how transport, persistence, and policy interact across several route families.

Security is built around SIWE-style wallet authentication, JWT access and refresh tokens, and API keys for long-lived agent access. JWT-only routes are reserved for actions bound closely to account state, such as API-key lifecycle and wallet rotation, while generic bearer routes accept either JWTs or API keys where appropriate. The platform also validates idempotency headers, applies rate limits, enforces payload-size limits, signs upstream provider requests, and blocks suspended or delisted services from sensitive operations.

The most distinctive runtime path is paid invocation. A consumer first discovers a service, inspects its schema and pricing, creates a quote for the exact payload, and then invokes the endpoint. If payment details are missing, the platform returns `402 Payment Required`. Once a valid payment payload is supplied, the service verifies and settles payment through the x402 integration, records the payment attempt, and only then forwards the request upstream. This choice is important because the intended clients are often agents rather than humans: they need a native, automatable way to pay for API access, whereas conventional web

checkout flows and card payments are poorly suited to autonomous machine-to-machine transactions. x402 therefore fits the marketplace model better than a traditional payment gateway bolted onto a REST API.

Figure 2. Paid invoke flow



6. Testing, Documentation, and Deployment

The repository shows a serious testing strategy rather than route-only checking. It contains 98 unit test files, 51 integration test files, 52 API test files, and 1 end-to-end test file. In the isolated worktree created for this report, a full baseline run completed successfully with `425 passed, 3 skipped`, with the only skips caused by Redis-dependent guardrail tests not being configured in the local environment. This gives strong evidence that the project was developed with verification in mind rather than only manual demonstration.

Documentation quality is also one of the stronger aspects of the repository. The project includes a README for setup and overview, a Markdown API reference, a committed API-reference PDF, an agent integration guide, a demo setup guide, and runnable example scripts for provider and consumer flows. This matters because the brief treats documentation as a core deliverable rather than an optional extra. In addition, the repository contains a Railway deployment runbook that documents environment variables, health checks, migrations on deploy, and smoke-test expectations, which supports the claim that the system is deployable in a realistic way even when demonstrated locally in the oral exam.

7. Challenges, Limitations, and Future Improvements

The most challenging part of the project is not simple CRUD but safe orchestration across asynchronous, stateful flows. Payment handling, quote validation, idempotent invoke behaviour, and provider forwarding all depend on the system preserving consistent state under retry conditions. Similarly, schema validation and request hashing have to match across quoting and invocation, otherwise the platform could either reject valid work or accept execution against stale pricing and contract data. Another important design challenge was preventing upstream leakage: provider routes must be expressive enough to configure forwarding targets, while public and consumer routes must never expose that internal routing data.

The main lesson learned is that clean layering matters most when the business flow becomes non-trivial. Keeping route handlers thin makes error handling and status-code mapping easier to reason about, while repository and service boundaries make persistence and orchestration testable in isolation. A second lesson is that failure-path testing is essential: retry behaviour, `402` responses, moderation blocks, payout conflicts, and invalid quote flows are the cases most likely to break user trust in a marketplace backend.

This implementation also has clear limitations. It is a backend-only system with no dedicated frontend. The brief's minimum CRUD requirement is met more convincingly across the repository as a whole than through a single public CRUD resource. Some advanced marketplace features, such as richer pricing models, broader observability, and more extensive end-to-end operational scenarios, remain future work rather than finished scope. The most useful next steps would be stronger e2e coverage, richer provider analytics, more payment and pricing flexibility, and a production-grade monitoring story beyond the current health and logging foundations.

8. Generative AI Declaration and Analysis

Generative AI was used as a structured development aid throughout this project and is declared here in line with the brief. Idea exploration and early discussion were carried out conversationally in ChatGPT web, while repository-based implementation and documentation work were supported with Codex agents. The workflow also included prompt-guided planning materials in the retained `codex-agent-plan/PROMPTS/` folder, skills-guided development workflows such as `using-superpowers`, `brainstorming`, and `writing-plans`, and AI-assisted reasoning for planning, architecture exploration, debugging support, drafting, and refinement.

A notable aspect of the workflow was its multi-agent structure. An orchestrator was used to coordinate specialised agents, each assigned one bounded task at a time. Those agents worked in isolated git worktrees and their outputs were then reviewed and merged back into the working branch. This separation reduced context pollution and generally improved the quality of the results because each agent only needed to reason about one problem rather than the full repository at once. In addition, each major phase began in plan mode so that the scope of the task, the intended outcome, and key implementation decisions were clarified before coding started.

The main benefit of AI in this project was speed with structure. ChatGPT web was useful for discussing ideas and exploring alternatives at a high level, while Codex agents were used for repository-aware coding and report-production tasks. The orchestrated agent workflow was particularly useful for decomposing planning, review, and implementation into smaller steps that could be checked independently. Across both tools, AI was useful for checking the implications of stack choices, refining documentation structure, and surfacing edge cases in areas such as payment flow, moderation, and testing. However, AI output was never accepted uncritically. Repository files, the coursework brief, and runtime documentation were used as the source of truth, and all final wording in this report was checked against the actual codebase before inclusion. Exported conversation logs are supplied in the appendix material for transparency.

Appendix Note

To keep the main body within the five-page target, the submission appendix is prepared separately in `docs/technical-report/technical-report-appendix.md`. That appendix includes the public deployment link, the reference list, and the exported GenAI conversation logs required by the brief.